สรุปภาพรวมงาน: ระบบรวบรวมราคาการ์ด One Piece Card Game ชุด OP17 จากหลายแหล่ง

02739452 แพลตฟอร์มข้อมูลขนาดใหญ่และการวิเคราะห์ · สอบปลายภาค ภาคต้น 2569
ชื่อ ป้องเกียรติ หันกลาง รหัสนิสิต 6621650582 หมู่ 820 ลำดับที่ 03
GitHub: https://github.com/arm89400-hue/02739452-03-6621650582

1. ภาพรวมงานแต่ละส่วน

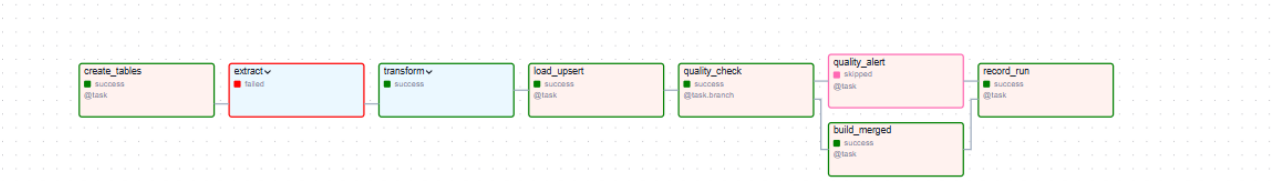
หัวข้อ: ราคาการ์ด One Piece Card Game ชุดล่าสุดที่ทุกแหล่งมีข้อมูล คือ OP17 "The World's Strongest Warriors" นำราคาจาก 3 เว็บมารวมเป็นชุดข้อมูลเดียว เพื่อเปรียบเทียบราคาการ์ดใบเดียวกันข้ามตลาด (ไม่มีข้อมูลส่วนบุคคล)

แหล่งข้อมูล	วิธีดึง	ข้อมูลที่ได้
PriceCharting	Web scraping (requests + BeautifulSoup, ไม่ใช้ Selenium) robots.txt อนุญาต	ราคา Ungraded / Grade 9 / PSA 10 (USD)
Limitless TCG	Web scraping หน้า set + หน้าการ์ดทีละใบ (delay 0.7 วินาที) robots.txt อนุญาตทั้งหมด	ราคา USD + EUR ของแต่ละเวอร์ชันการ์ด
TCGplayer ผ่าน TCGCSV	JSON API (products + prices) ไม่ต้องใช้ key	Market price (USD), rarity, เลขการ์ด

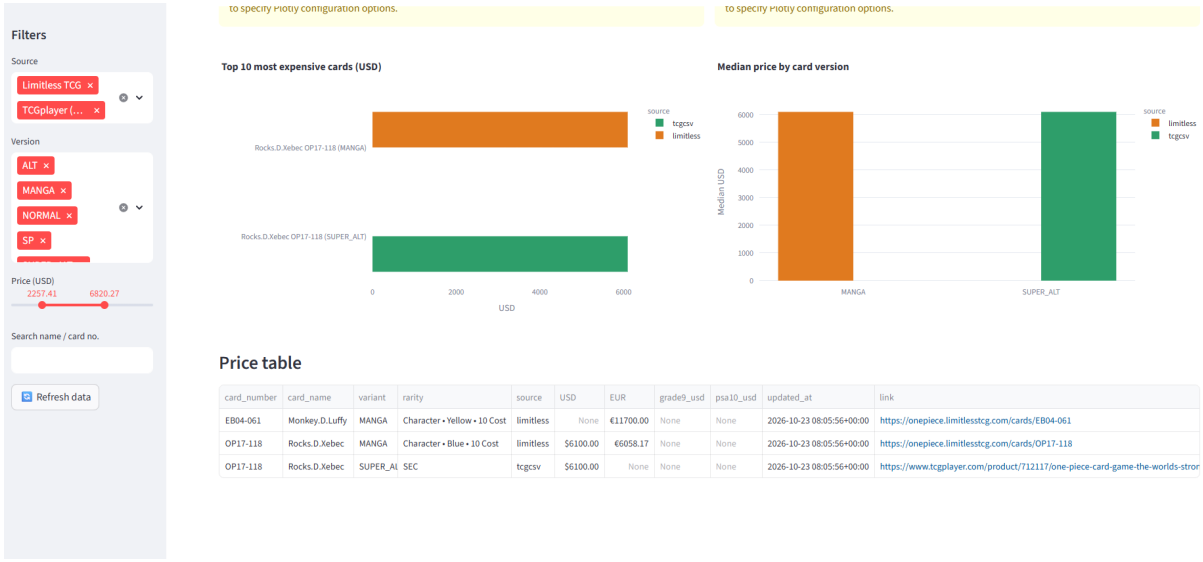
สถาปัตยกรรม: Docker Compose → Apache Airflow 2.10 (LocalExecutor) → PostgreSQL 16 → เว็บ dashboard Streamlit (อ่านจาก PostgreSQL โดยตรง)

DAG op17_card_prices: create_tables → TaskGroup extract (3 แหล่งรันขนาน) → TaskGroup transform (Dynamic Task Mapping) → load_upsert → quality_check (branch) → build_merged / quality_alert → record_run

ระดับ	สิ่งที่ทำได้	สถานะ
ระดับ 1	DAG extract → transform → load ลง PostgreSQL, เว็บแสดงตารางและกราฟ	✔️ ครบ
ระดับ 2	3 แหล่งรันขนานใน TaskGroup, รวมเป็น schema เดียว, UPSERT (ON CONFLICT (source, source_key) DO UPDATE) รันซ้ำแล้วจำนวนแถวไม่เพิ่ม, เว็บมี KPI 5 ตัว และตัวกรอง (แหล่ง / เวอร์ชัน / ช่วงราคา / ค้นหา)	✔️ ครบ
ระดับ 3	transform/load ใช้ trigger_rule=all_done → แหล่งใดล่มยังรวมแหล่งที่เหลือได้ (ทดสอบด้วย param simulate_down), ตัดรายการซ้ำข้ามแหล่งด้วยคีย์ (เลขการ์ด + เวอร์ชัน) ลงตาราง cards_merged, quality_check ตรวจ 5 ข้อ แล้ว branch ไป build_merged หรือ quality_alert	✔️ ครบ
โบนัส	หน้า Pipeline status บนเว็บ (สถานะทุก task, ผลตรวจคุณภาพ, ประวัติการรัน) และใช้ Dynamic Task Mapping	✔️ ทั้งสองอย่าง



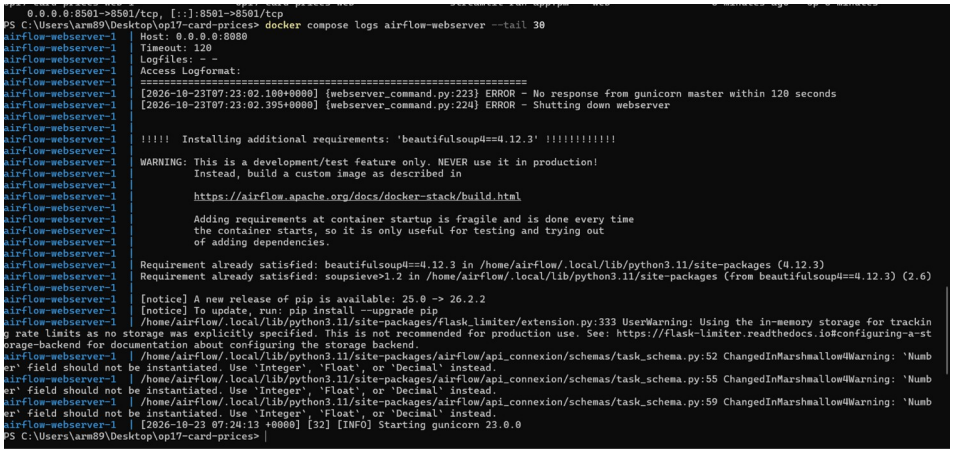
ภาพที่ 1: Graph view ของ DAG op17_card_prices จากการรันจริง — extract (PriceCharting) ล้มเหลว แต่ transform, load_upsert, quality_check และ build_merged ยังทำงานต่อจนสำเร็จ (ระดับ 3: แหล่งล่มแล้วรวมแหล่งที่เหลือต่อได้), quality_alert ถูกข้ามเพราะตรวจคุณภาพผ่าน



ภาพที่ 2: หน้าเว็บ dashboard (Streamlit อ่านจาก PostgreSQL) — ตัวกรองแหล่ง/เวอร์ชัน/ช่วงราคา/ค้นหา, กราฟ 10 อันดับราคาสูงสุด, ราคามัธยฐานตามเวอร์ชัน และตารางราคา

2. ปัญหาที่พบและวิธีแก้

ปัญหา	อาการ	วิธีหาสาเหตุ	วิธีแก้
1. Airflow webserver เปิดไม่ขึ้น	เปิด localhost:8080 ได้ ERR_EMPTY_RESPONSE แม้ container สถานะ Up	docker compose logs airflow-webserver พบ "No response from gunicorn master within 120 seconds" แล้ว webserver ถูก restart วนซ้ำ (เครื่องช้า เปิด 4 workers + ติดตั้ง package ตอน start)	ตั้ง WEB_SERVER_MASTER_TIMEOUT=600, WORKERS=2, ไม่ให้ webserver ติดตั้ง package เพิ่ม แล้ว docker compose up -d --force-recreate airflow-webserver
2. Limitless ไม่มีราคาในหน้ารวม	หน้า set ของ Limitless แสดงแค่ราคารวมทั้งชุด ไม่มีราคาการ์ดรายใบ	ตรวจโครงสร้างหน้าเว็บ พบว่าราคาอยู่ในหน้ารายละเอียดของการ์ดแต่ละใบ (USD/EUR แยกตามเวอร์ชัน)	ดึงลิงก์การ์ดจากหน้า set แล้ว scrape ทีละหน้า หน่วงเวลา 0.7 วินาทีเพื่อไม่ให้โหลดเว็บหนัก หน้าใดพลาดข้ามไป ไม่ทำให้ทั้งแหล่งล้ม
3. ตัดรายการซ้ำข้ามแหล่งไม่ได้	การ์ดใบเดียวกันจับคู่กันไม่ติด เพราะแต่ละเว็บเขียนเวอร์ชันต่างกัน	เทียบข้อมูลดิบ: PriceCharting ใช้ [Super Alternate Art], TCGplayer ใช้ (Alternate Art), Limitless ใช้ตัวย่อ aa / sp	ดึงเลขการ์ดด้วย regex (เช่น OP17-079) และสร้างฟังก์ชัน norm_variant แปลงเป็นรหัสกลาง NORMAL/ALT/SUPER_ALT/MANGA/SP แล้วใช้ (card_number, variant) เป็นคีย์
4. ชุดล่าสุด OP18 ไม่ครบทุกแหล่ง	หา OP18 ไม่เจอใน PriceCharting และ Limitless	ตรวจรายการชุดของทั้ง 3 เว็บ: OP18 มีแค่ใน TCGCSV	เลือก OP17 ซึ่งเป็นชุดล่าสุดที่ทั้ง 3 แหล่งมีข้อมูล เพื่อให้ตัดรายการซ้ำข้ามแหล่งได้จริง



ภาพที่ 3: log ของ airflow-webserver ที่ใช้หาสาเหตุปัญหาที่ 1 (No response from gunicorn master within 120 seconds)

3. การวางแผนและจัดลำดับงาน

ช่วงเวลา	งาน
13:00–13:40	อ่านโจทย์ เลือกหัวข้อ ตรวจแหล่งข้อมูล (robots.txt, ต้องไม่ใช้ Selenium, มีข้อมูล ≥ 100 รายการ) แล้วจองหัวข้อในไลน์
13:40–14:00	เขียน DAG + ฟังก์ชัน extract/transform/load, docker-compose, dashboard แล้วทดสอบ DAG กับ PostgreSQL ก่อน push ขึ้น GitHub (commit แรก 13:55)
14:00–14:50	รันระบบด้วย Docker บนเครื่อง แก้ปัญหา webserver, ตรวจผลการดึงข้อมูลจริง
14:50–15:20	ย้ายไปเครื่องใหม่ (clone จาก GitHub + สร้าง .env) รัน DAG ทั้งแบบปกติและ simulate_down เก็บภาพหน้าจอ
15:20–16:00	เขียน PDF สรุป ตรวจ README และ commit สุดท้ายก่อน 16:00

หลักการจัดลำดับ: ทำระบบให้ครบทุกระดับแบบเรียบง่ายก่อน (end-to-end) แล้วค่อยแก้ปัญหาตอนรันจริง เลือกแหล่งที่เป็น HTML ธรรมดาหรือ API ฟรี เพื่อไม่เสียเวลากับเว็บที่ต้องใช้ JavaScript และ commit ขึ้น GitHub เร็วเพื่อกันงานหาย

งานที่ตัดออกและเหตุผล:

- **เก็บประวัติราคาตามเวลา (price history):** UPSERT จะเขียนทับราคาเดิม ถ้าจะเก็บต้องมีอีกตาราง และใน 3 ชั่วโมงมีข้อมูลแค่วันเดียว กราฟแนวโน้มจึงไม่มีความหมาย
- **ร้านค้าไทย / เว็บ TCGplayer โดยตรง:** หน้าเว็บโหลดด้วย JavaScript ต้องใช้ Selenium ซึ่งโจทย์ห้าม จึงใช้ TCGCSV (ข้อมูล TCGplayer แบบ JSON) แทน
- **แจ้งเตือนผ่าน LINE/Email เมื่อ quality ไม่ผ่าน:** ต้องมี token ภายนอก จึงทำเป็น task quality_alert ที่บันทึก log และแสดงผลบนหน้า Pipeline status แทน
- **แปลงราคาเป็นเงินบาท:** ต้องเพิ่มแหล่งอัตราแลกเปลี่ยนอีกตัว ไม่ใช่ประเด็นหลักของโจทย์ จึงแสดงเป็น USD/EUR ตามแหล่งเดิม